Milestone 2 Project Outline Requirement:

1. Motivation for the idea/description of the problem the application solves

Choosing a flight involves more than finding the cheapest ticket. Travelers also consider when to book their tickets to get the best prices and how reliable their flights will be. However, information about ticket prices, airline performance, airport operations, and weather conditions is often scattered across different sources, making it difficult to compare options and make informed decisions.

Our application aims to address this problem by bringing together historical flight prices, flight performance records, and weather data in one platform. Users can compare flight options, explore ticket price trends, and learn more about airlines and airports, including their route networks, operational performance, and seasonal weather patterns. By providing these insights in an accessible way, our website helps travelers make more informed travel decisions based on cost, convenience, and reliability.

2. List of features you will definitely implement in the application

A. Flight Search and Comparison: Users can search for flights by departure airport, destination, and travel date. They can compare flight duration, and historical delay and cancellation rates. Users can also filter results by price range, departure time, airline and ticket type (e.g., Basic Economy or refundable tickets), and sort flights by price, duration, or historical reliability.
B. Ticket Price Trends: Users can explore historical ticket prices across different booking windows (e.g., 1–3 days, 3–7 days, and more than 7 days before departure) to identify potentially cheaper booking periods.
C. Airline Network and Pricing Information: Users can explore airline information, including the number of cities and airports served, available routes, most frequently operated routes, and average ticket prices.
D. Airline Reliability Analysis: Users can view historical airline performance statistics, including delay rates, cancellation rates, average delay duration, and common causes of delays.
E. Airport Traffic and Connectivity: Users can explore airport information, including flight volume, number of airlines served, number of destinations, busiest routes, and average taxi times.
F. Seasonal Weather and Flight Reliability: Users can explore monthly weather patterns at different airports, including average temperature, precipitation, and snowfall, alongside historical delay and cancellation rates to identify periods with higher risks of flight disruptions.

3. **List of features you might implement in the application, given enough time**

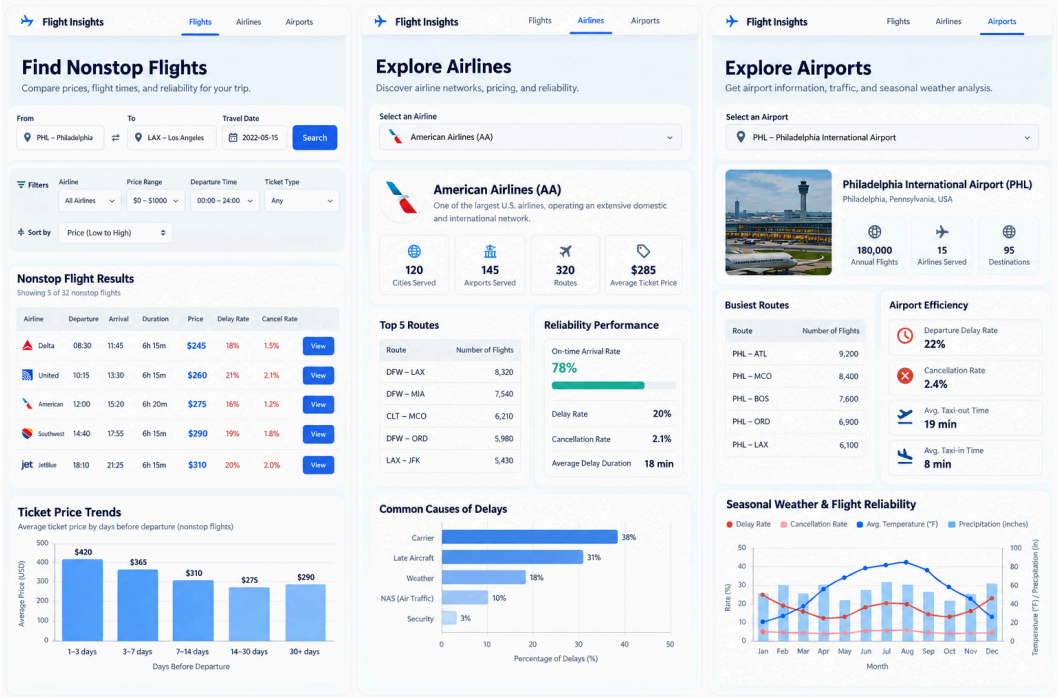
A. Map Exploration: Users can select a departure airport and explore available nonstop flight routes to different destinations on an interactive map. They can click on a destination to view route details, including airlines operating the route, average ticket prices, and historical flight reliability.

4. **List of pages the application will have and a 1-2 sentence description of each page. We expect that the functionality of each page will be meaningfully different than the functionality of the other pages.**

A. Flights Page: Users can search for and compare flights based on ticket prices, flight duration, and historical reliability. The page also provides historical ticket price trends across different booking windows to help users decide which flights to choose and when to book.

B. Airlines Page: Users can explore airline profiles, including their route networks, number of destinations served, busiest routes, and average ticket prices. The page also provides historical performance statistics, such as delay rates, cancellation rates, and common causes of delays.

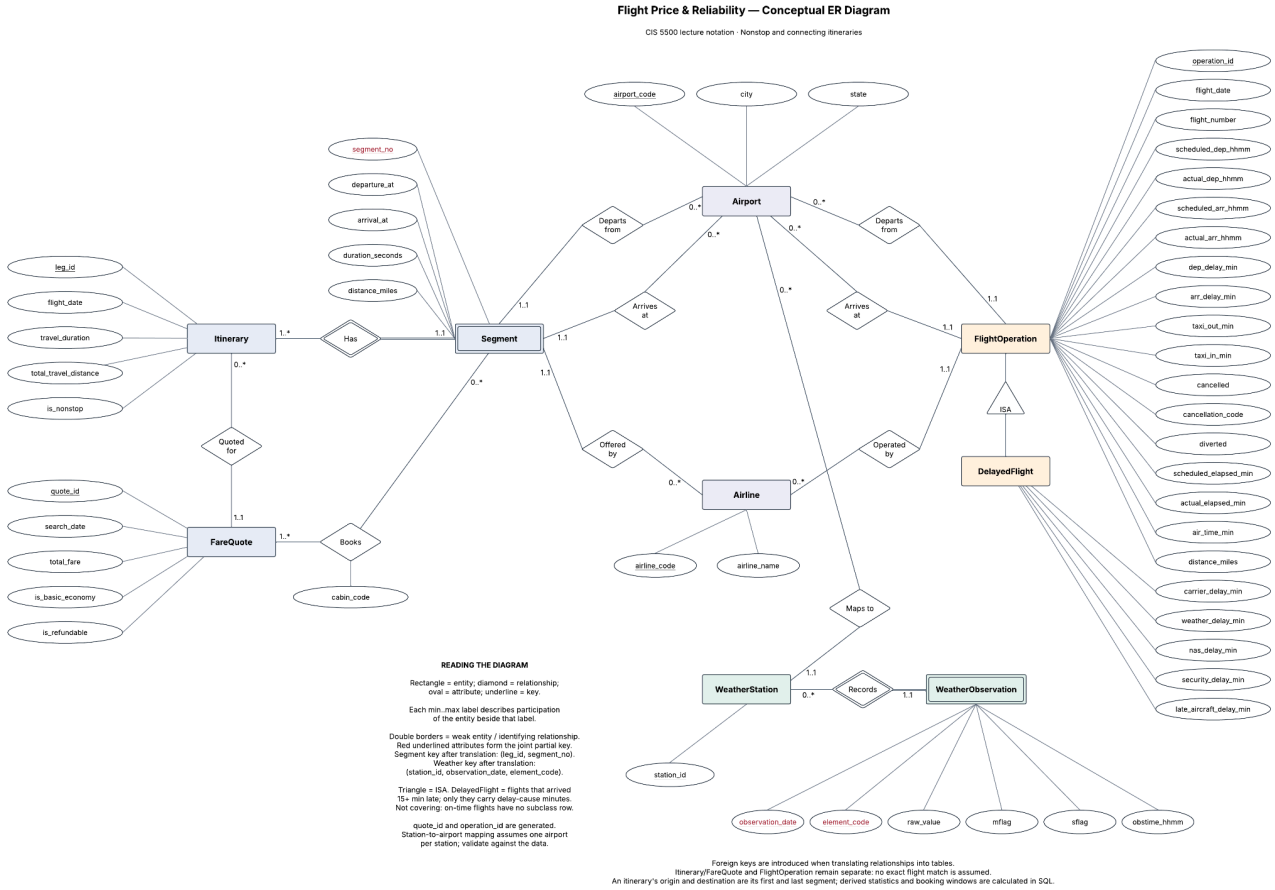
C. Airports Page: Users can explore airport traffic and connectivity, including flight volume, airlines served, popular destinations, and average taxi times. The page also presents seasonal weather patterns alongside historical delay and cancellation rates to help users understand how flight reliability varies throughout the year.



D. Map Exploration Page (Optional): Users can select a departure airport to visualize its connected destinations and nonstop flight routes on an interactive map. By selecting a route, users can explore flight information, including operating airlines, average ticket prices, and historical delay and cancellation rates.

5. Relational schema as an ER diagram

The ER diagram below illustrates the overall database design for our application, including the main entities, their attributes, and the relationships between them. A clear version of ER diagram is available here: Flight Project ER Diagram.png



Corresponding to the ER diagram above, we have the following **relational schema**. Primary keys are underlined, and foreign key constraints are specified below the corresponding relations.

Airport (airport_code, city, state)

Airline (airline_code, airline_name)

Itinerary (leg_id, flight_date, travel_duration, total_travel_distance, is_nonstop)

Segment (leg_id, segment_no, departure_at, arrival_at, duration_seconds, distance_miles, origin_code, destination_code, airline_code)

leg_id FOREIGN KEY REFERENCES Itinerary (leg_id)
origin_code FOREIGN KEY REFERENCES Airport (airport_code)
destination_code FOREIGN KEY REFERENCES Airport (airport_code)
airline_code FOREIGN KEY REFERENCES Airline (airline_code)

FareQuote (quote_id, search_date, total_fare, is_basic_economy, is_refundable, leg_id)

leg_id FOREIGN KEY REFERENCES Itinerary (leg_id)

Books (quote_id, leg_id, segment_no, cabin_code)

quote_id FOREIGN KEY REFERENCES FareQuote (quote_id)
(leg_id, segment_no) FOREIGN KEY REFERENCES Segment (leg_id, segment_no)

FlightOperation (operation_id, flight_date, flight_number, scheduled_dep_hhmm, actual_dep_hhmm, scheduled_arr_hhmm, actual_arr_hhmm, dep_delay_min, arr_delay_min, taxi_out_min, taxi_in_min, cancelled, cancellation_code, diverted, scheduled_elapsed_min, actual_elapsed_min, air_time_min, distance_miles, origin_code, destination_code, airline_code)

origin_code FOREIGN KEY REFERENCES Airport (airport_code)
destination_code FOREIGN KEY REFERENCES Airport (airport_code)
airline_code FOREIGN KEY REFERENCES Airline (airline_code)

DelayedFlight (operation_id, carrier_delay_min, weather_delay_min, nas_delay_min, security_delay_min, late_aircraft_delay_min)

operation_id FOREIGN KEY REFERENCES FlightOperation (operation_id)

WeatherStation (station_id, airport_code)

airport_code FOREIGN KEY REFERENCES Airport (airport_code)

WeatherObservation (station_id, observation_date, element_code, raw_value, mflag, sflag, obstime_hhmm)

station_id FOREIGN KEY REFERENCES WeatherStation (station_id)

6. SQL DDL for creating the database

The SQL DDL statements for creating the database tables, including primary key and foreign key constraints, are provided in the following file: schema.sql

7. Explanation of how you will clean and pre-process the data. This tutorial demonstrates how to do simple pre-processing in Python.

We will use Python and pandas to inspect and preprocess the datasets before loading them into PostgreSQL. Specific cleaning rules will be refined after examining missing values, duplicates, and inconsistent records.

- **Flight prices**: We will process the large dataset in chunks and validate dates, fares, and itinerary identifiers. We will split the || -separated fields into ordered flight segments and check that the segment information aligns. Separate fare observations across search dates and ticket types will be preserved, with cabin information linked to the appropriate quote and segment.
- **Flight performance**: We will standardize airport and airline identifies and validate dates, local time fields, delay values, and status indicators. Negative delays will be preserved because they indicate early departures or arrivals. Cancelled and diverted flights will remain in the dataset, and missing actual times or delays will remain null. We will check duplicate operation records and the completeness of delay-cause values.
- **Weather**: We will verify airport-to-station mappings, inspect quality flags, and identify missing or invalid measurements. Invalid or missing measurements will be excluded from the observation table rather than replaced with zeros. Weather observations will remain in long format, with one record per station, date, and element. We will preserve raw measurement values, measurement flags, source flags, and observation times.
- **Cross-dataset integration:** We will standardize shared airport and airline codes to support joins across datasets. Flight performance and weather records will be linked by airport and date, while flight prices (2022) and performance records (2025) will be compared at aggregated airline or route levels

8. **List of technologies you will use. You must use some kind of SQL database. We recommend using PostgreSQL specifically because you will use PostgreSQL in HW2, and we will provide guidance for setting up a PostgreSQL database.**

   - **Frontend:** React, JavaScript, HTML, CSS, Material UI (MUI)
     — Build the interactive web interface.
   - **Backend:** Node.js
     — Handle API requests and execute SQL queries.
   - **Database:** PostgreSQL
     — Store and manage flight pricing, flight performance, and weather data.
   - **Data Processing:** Python
     — Clean, preprocess, and transform datasets.
   - **Cloud Hosting:** Amazon Web Services (AWS)
     — Use Amazon RDS for PostgreSQL database hosting.
   - **Version Control:** Git and GitHub
     — Manage source code and support team collaboration.

9. **Description of what each group member will be responsible for**

Task	**Group Member**
Data Cleaning & Preprocessing – Clean and prepare datasets	Crystal Wang, Shihan Li
Database Setup – Set up PostgreSQL, create tables	Shihan Li, Sabina Lu
Backend & SQL Queries – Implement SQL queries and APIs using Node.js	Peggy Che, Shihan Li, Sabina Lu
Frontend Development – Build Flights, Airlines, and Airports pages	Peggy Che, Crystal Wang
Data Visualization – Implement charts and potentially the interactive map	Crystal Wang
Deployment & Testing – Deploy the application, integrate components, and test functionality	Sabina Lu, Peggy Che